ACADEMIC YEAR: 2025-26

Course: Skill base Lab Course: Cloud Computing

Course code: CSL605

Year/Sem: TE/VI

Experiment No.: 01
Aim: To study and implement Infrastructure as a Service using AWS EC2 by creating Ubuntu Virtual Machine through PUTTY open Source terminal emulator
Name: Shravani Sandeep Raut
Roll Number: 40
Date of Performance: 05/01/2026
Date of Submission: 07/01/2026

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	**20**	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	4-5	2-3	2
Understanding	4-5	2-3	2
Journal work and timely submission.	8-10	5-8	4

Checked by

Name of Faculty : Dr. Tatwadarshi Nagarhalli

Signature :

Date :

Experiment No. 1

Aim: To study and Implement Infrastructure as a Service using AWS EC2 by creating a Ubuntu Virtual Machine through PuTTY open source terminal emulator

Objective: To demonstrate the steps to create and run virtual machines inside Amazon EC2

Theory:

IAAS: Infrastructure as a service (IaaS) is a type of cloud computing service that offers essential compute, storage, and networking resources on demand, on a pay-as-you-go basis. IaaS is one of the four types of cloud services, along with software as a service (SaaS), platform as a service (PaaS), and serverless.

In an IaaS service model, a cloud provider hosts the infrastructure components that are traditionally present in an on-premises data center. This includes servers, storage and networking hardware, as well as the virtualization or hypervisor layer.

IaaS providers also supply a range of services to accompany those infrastructure components.

IaaS customers access resources and services through a wide area network (WAN), such as the internet, and can use the cloud provider's services to install the remaining elements of an application stack. For example, the user can log in to the IaaS platform to create virtual machines (VMs); install operating systems in each VM; deploy middleware, such as databases; create storage buckets for workloads and backups; and install the enterprise workload into that VM. Customers can then use the provider's services to track costs, monitor performance, balance network traffic, troubleshoot application issues and manage disaster recovery.

AWS: AWS enables you to select the operating system, programming language, web application platform, database, and other services you need. With AWS, you receive a virtual environment that lets you load the software and services your application requires. This eases the migration process for existing applications while preserving options for building new solutions.
You pay only for the compute power, storage, and other resources you use, with no long-term contracts or up-front commitments.
AWS is designed to allow application providers, ISVs, and vendors to quickly and securely host your applications – whether an existing application or a new SaaS-based application.

EC2: Amazon Elastic Compute Cloud (Amazon EC2) provides on-demand, scalable computing capacity in the Amazon Web Services (AWS) Cloud. Using Amazon EC2 reduces hardware costs so you can develop and deploy applications faster. You can use Amazon EC2 to launch as many or as few virtual servers as you need, configure security and networking, and manage storage. You can add capacity (scale up) to handle compute-heavy tasks, such as monthly or yearly processes, or spikes in website traffic. When usage decreases, you can reduce capacity

(scale down) again.

Steps:

- We will start by creating an EC2 instance. Go to EC2 Console https://console.aws.amazon.com/ec2/v2/home and Click on Launch instance. Where you will find multiple AMIs, Scroll down and select Ubuntu Server

- Once you select the server, follow further steps, and launch. Make sure while launching download the PEM file as it will need in the future.

- Now go to the EC2 dashboard and look for a public IP address and Copy that as we will need for PUTTY configuration, Though you can use Public DNS as well.

- We are done with AWS configuration, now open PUTTY and follow a few steps:

- Add copied IP address in the Host Name field and name the Saved session and save it. Will keep port as it is i.e., 22 as we have only opened the 22 port as we want everything should go through the SSH tunnel; though you can open other ports as well there will be less security.
- Add SSH (PPK) key by browsing SSH -> Auth -> Private Key file for authentication -> Browse and again go to Session and Save it.
  Note: By default, AWS only provide a PEM file. We need to convert PEM to PPK. You can find multiple online tools for converting this

- Click on Open, and it will prompt a security alert just choose Yes, and you will able to see the command prompt.
  On the command prompt, just log in as Ubuntu, as we have added PPK so it won't ask for Password.
- Once you get logged into the server, you need to follow the below.
  - sudo apt update && sudo apt upgrade
  - sudo sed -i 's/^PasswordAuthentication no/PasswordAuthentication yes/' /etc/ssh/sshd_config – To set login credentials
  - sudo /etc/init.d/ssh restart
  - sudo passwd ubuntu – This will be your login password to ubuntu machine
  - sudo apt install xrdp xfce4 xfce4-goodies tightvncserver
  - echo xfce4-session> /home/ubuntu/.xsession
  - sudo cp /home/ubuntu/.xsession /etc/skel
  - sudo sed -i '0,/-1/s//ask-1/' /etc/xrdp/xrdp.ini
  - sudo service xrdp restart

- Note: In between, if this process asks for some kind of choice, then select the local version installed option.

- Now, we need to configure the tunnel on PUTTY to the route of the request.

- Open PUTTY again and go to the Connection – > SSH -> Tunnels ->

1. Add Source Port (This could be any in my case I have added 8888).
2. Add Destination Port – This will be your instance private IP address following with 3389.
3. Click on Add

- Once done, now click on the open button and you will see a Command prompt where login Id will be Ubuntu.
  Now you can check either we are listening on the same port or not using the following command: netstat -antp

- Now we are done with the setup.

- Open Remote Desktop Connection

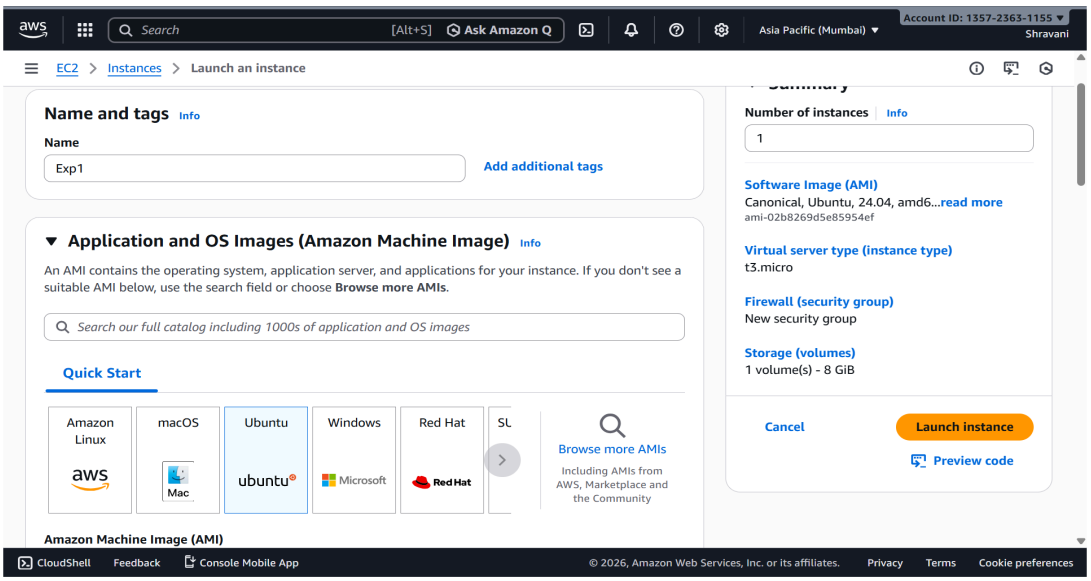
- Add localhost:8888 or 127.0.0.1:8888 and Connect

- You will be asked for a username and password; So username could be Ubuntu and Password will be added by you while installing the command. And Yeah, there you go, We have successfully logged into the Remote server.

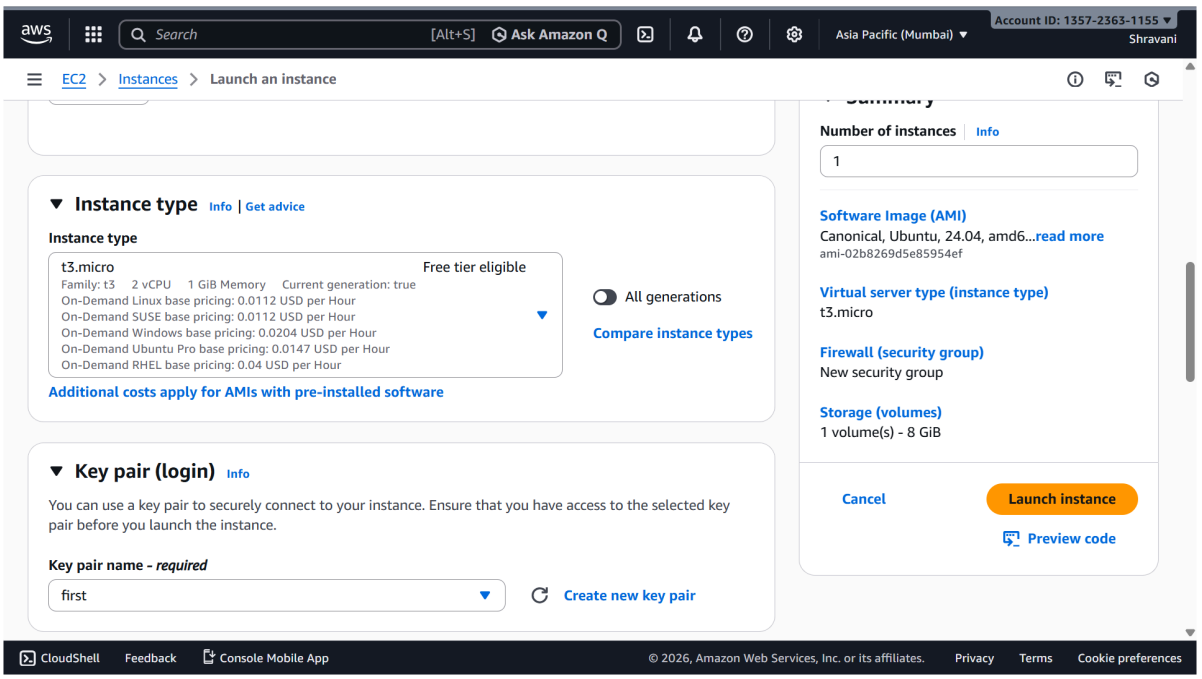
Output and Observation:

1. Go to AWS Dashboard > EC2 > Instances. Then click on Create Instance.

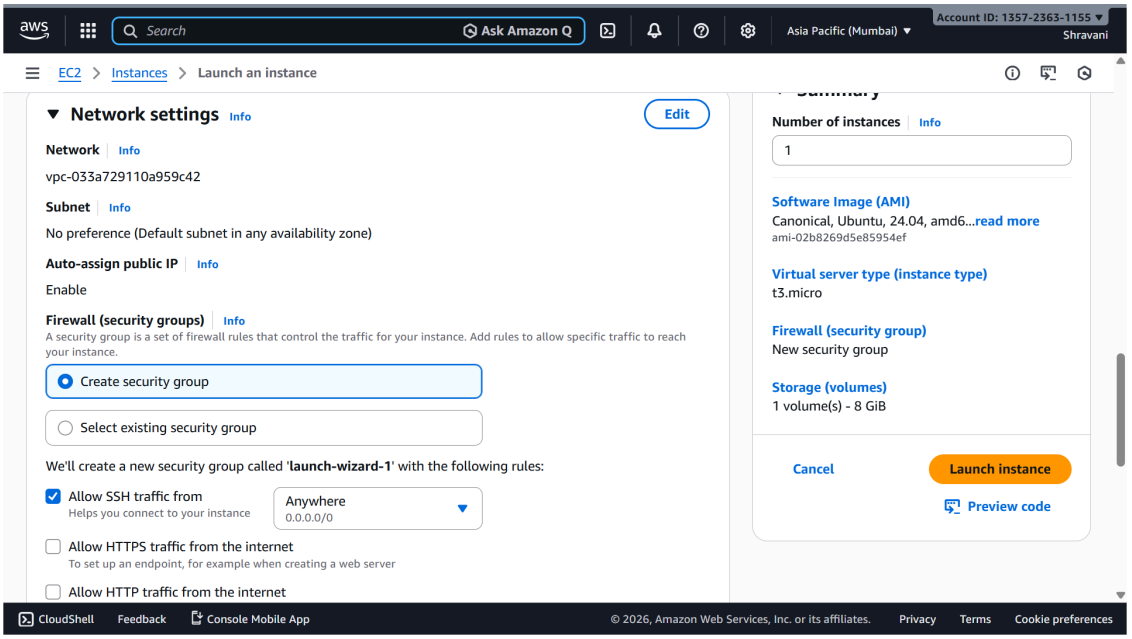
2. Enter the name of the Instance. *(Here, Exp1).* And Select the AMI as **Ubuntu.**



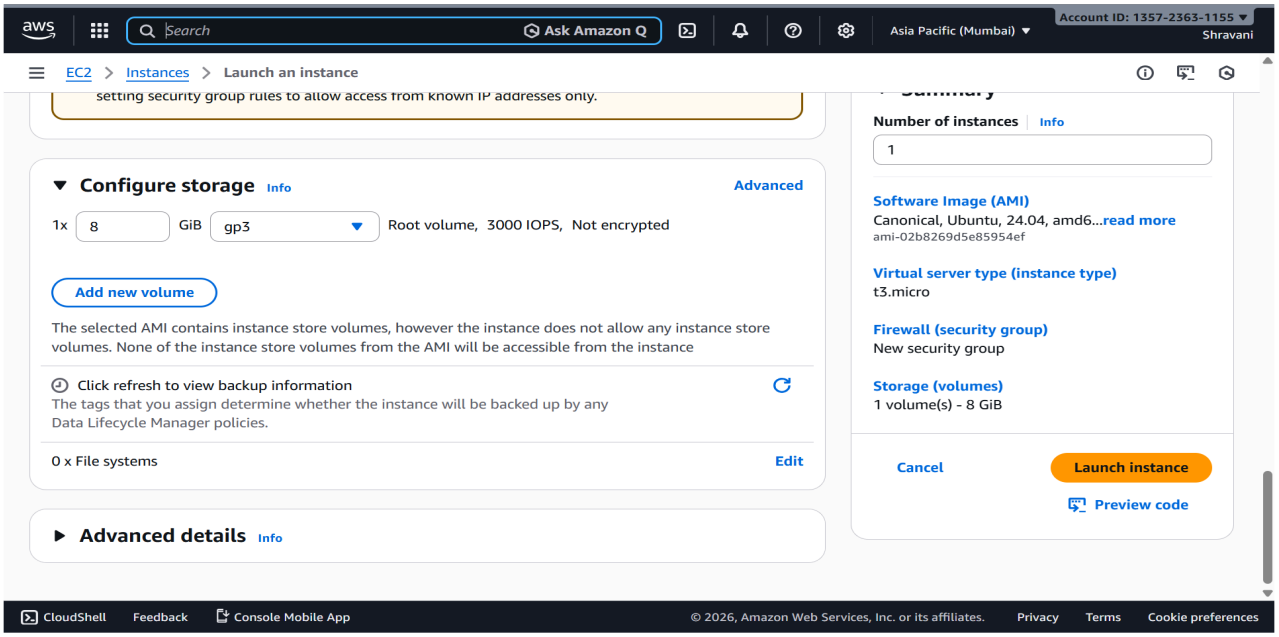
3. Select the Instance Type as t3.micro (or any other free tier eligible). Also Create a Key pair name and make sure you download and save it for further use.



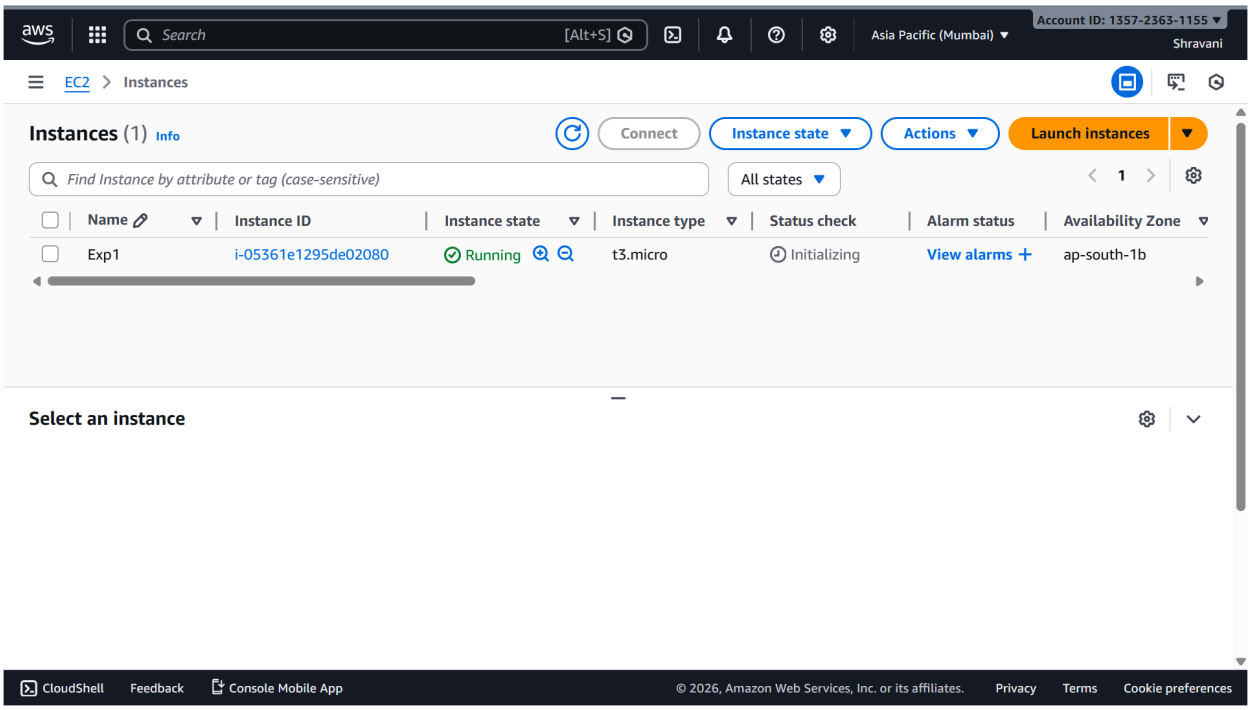
4. Go to the network settings tab and Allow SSH traffic from Anywhere



5. Do not change any settings in Configure Storage Tab

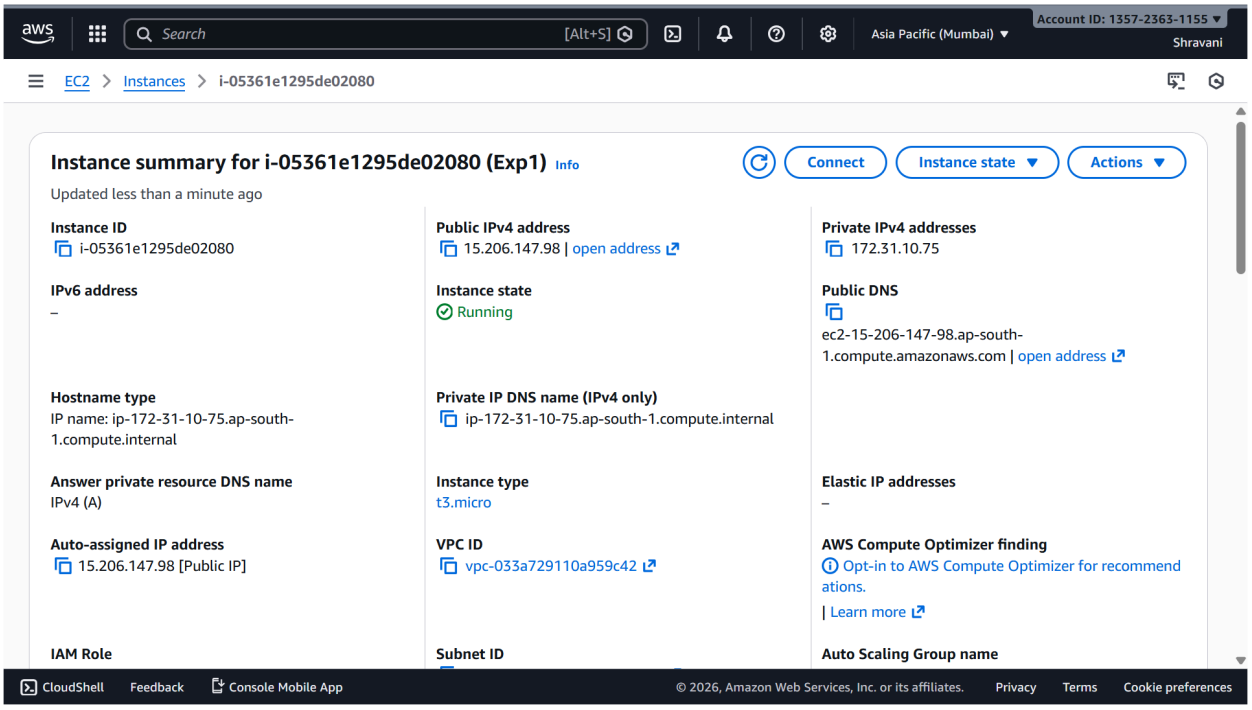


6. After this click on **Launch Instance.**



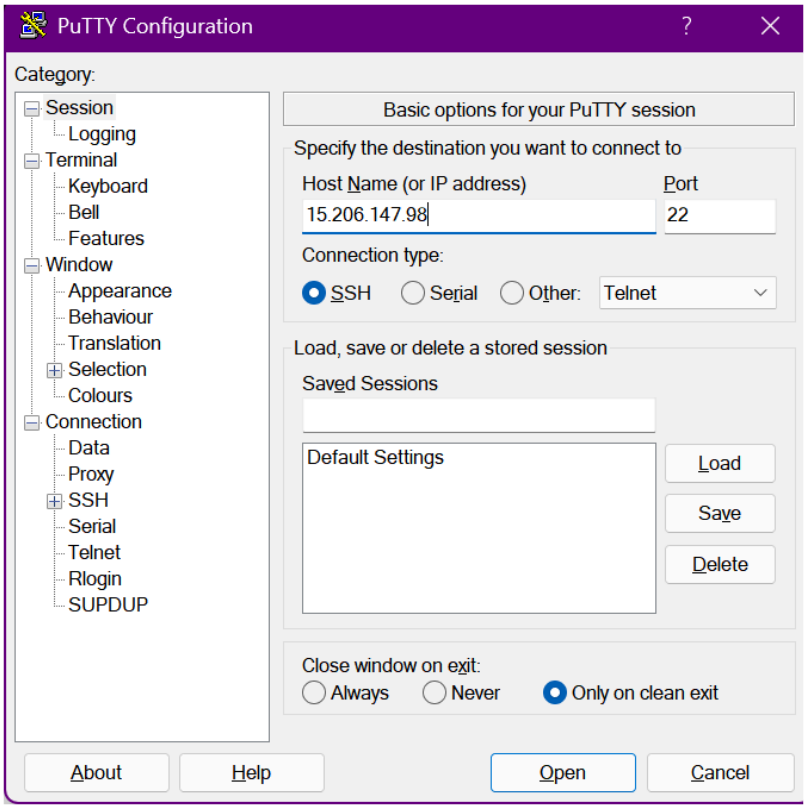
7. The below screenshot shows how the screen will look once the instance is created.

8. Click on the Instance ID to get more information about the instance.

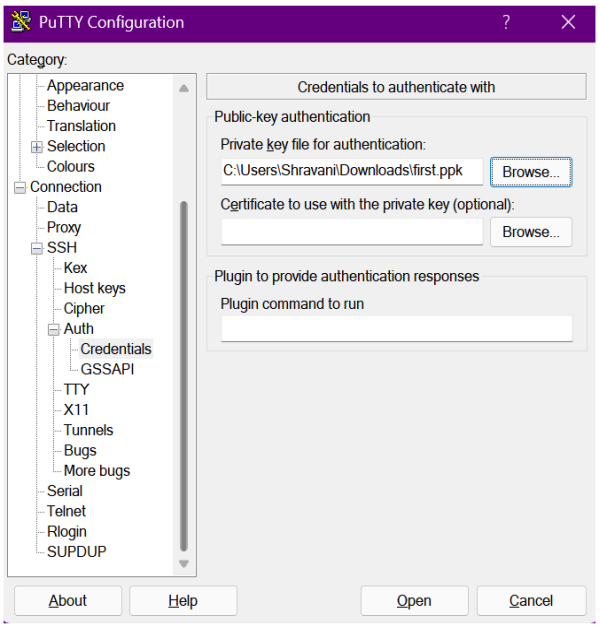


9. Download PuTTY from the official site [PuTTY Download]
10. Once installed this type of dialog box will appear.

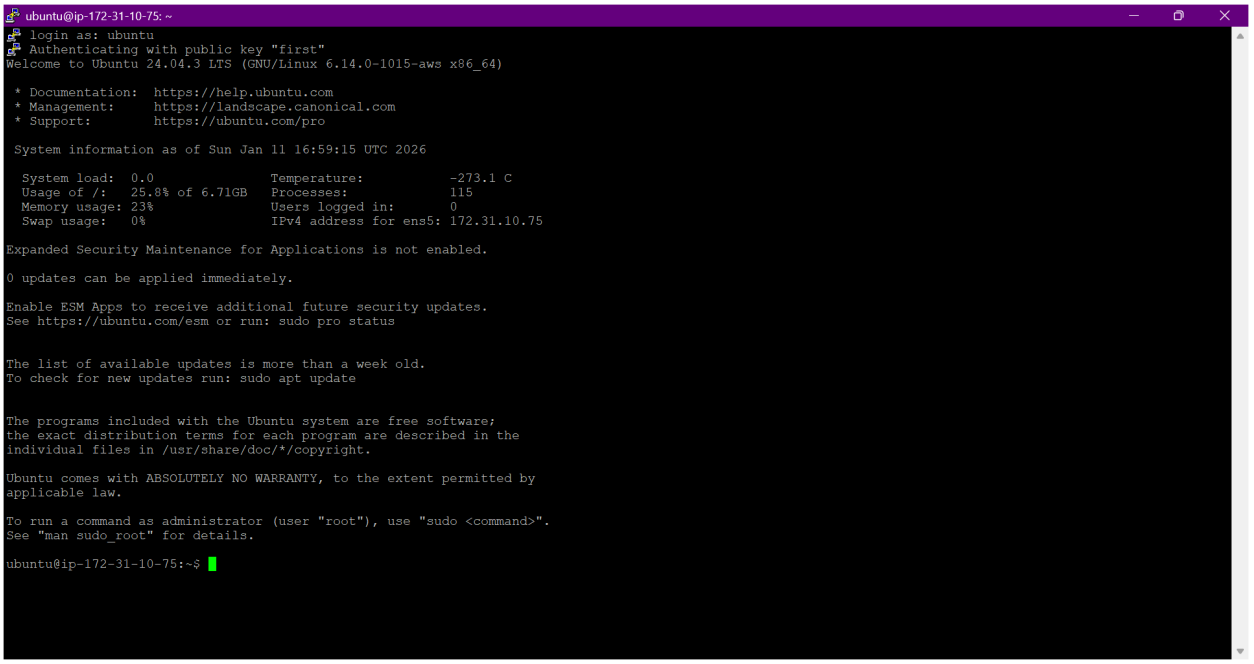
11. In place of Host Name put the **Public IP address** which u will get from the Instance Dashboard



12. Go to Connection > SSH > Auth > Credentials. Add tthe location where u stored ur private key



13. A SSH terminal will open.



14. Run commands

- **3.1 Check System Information**

  uname -a

- **3.2 Update Ubuntu Instance**

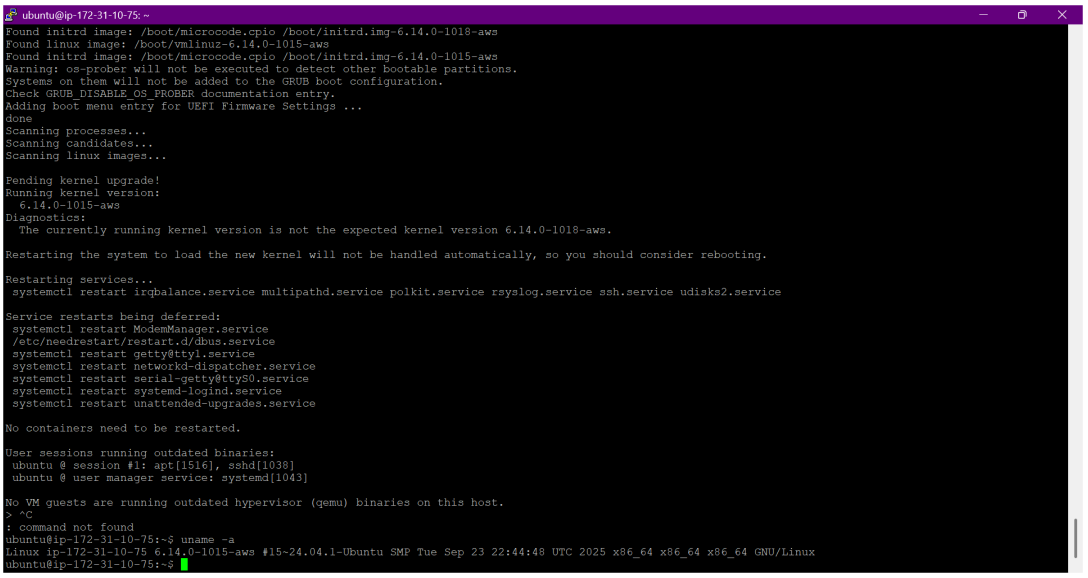
  sudo apt update && sudo apt upgrade -y

- **3.3 Install Apache Web Server**

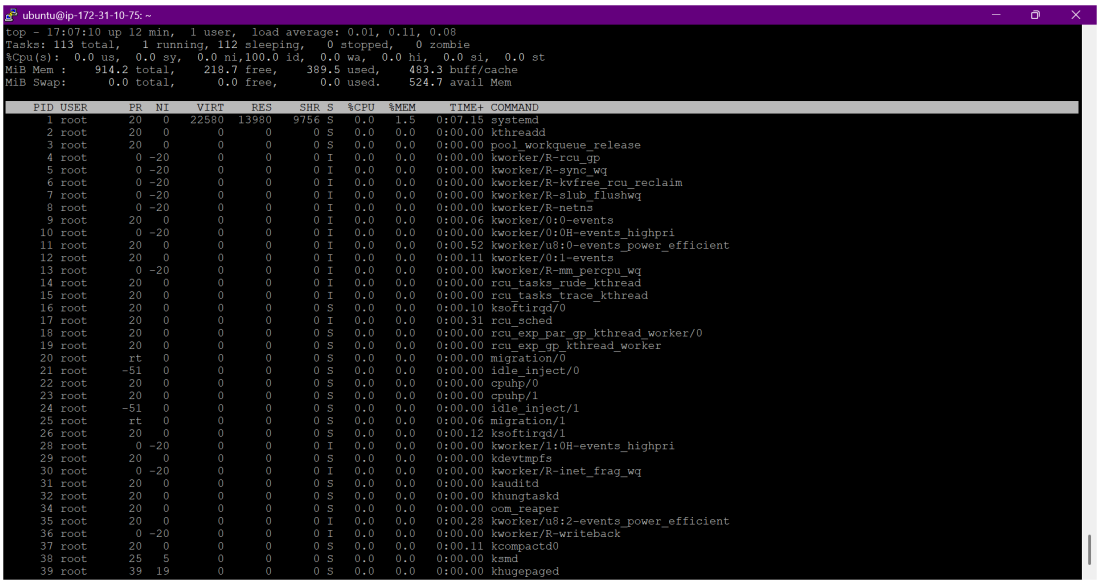
  sudo apt install apache2 -y

- 3.4 Verify Apache Installation

sudo systemctl status apache2



```
ubuntu@ip-172-31-10-75: ~
Found initrd image: /boot/microcode.cpio /boot/initrd.img-6.14.0-1018-aws
Found linux image: /boot/vmlinuz-6.14.0-1015-aws
Found initrd image: /boot/microcode.cpio /boot/initrd.img-6.14.0-1015-aws
Warning: os-prober will not be executed to detect other bootable partitions.
Systems on them will not be added to the GRUB boot configuration.
Check GRUB_DISABLE_OS_PROBER documentation entry.
Adding boot menu entry for UEFI Firmware Settings ...
done
Scanning processes...
Scanning candidates...
Scanning linux images...

Pending kernel upgrade!
Running kernel version:
  6.14.0-1015-aws
Diagnostics:
  The currently running kernel version is not the expected kernel version 6.14.0-1018-aws.

Restarting the system to load the new kernel will not be handled automatically, so you should consider rebooting.

Restarting services...
 systemctl restart irqbalance.service multipathd.service polkit.service rsyslog.service ssh.service udisks2.service

Service restarts being deferred:
 systemctl restart ModemManager.service
 /etc/needrestart/restart.d/dbus.service
 systemctl restart getty@tty1.service
 systemctl restart networkd-dispatcher.service
 systemctl restart serial-getty@ttyS0.service
 systemctl restart systemd-logind.service
 systemctl restart unattended-upgrades.service

No containers need to be restarted.

User sessions running outdated binaries:
 ubuntu @ session #1: apt[1516], sshd[1038]
 ubuntu @ user manager service: systemd[1043]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
> ^C
: command not found
ubuntu@ip-172-31-10-75:~$ uname -a
Linux ip-172-31-10-75 6.14.0-1015-aws #15~24.04.1-Ubuntu SMP Tue Sep 23 22:44:48 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
ubuntu@ip-172-31-10-75:~$
```



```
ubuntu@ip-172-31-10-75: ~
top - 17:07:10 up 12 min,  1 user,  load average: 0.01, 0.11, 0.08
Tasks: 113 total,   1 running, 112 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni,100.0 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem :    914.2 total,    218.7 free,    389.5 used,    483.3 buff/cache
MiB Swap:      0.0 total,      0.0 free,      0.0 used.    524.7 avail Mem

    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
      1 root      20   0   22580  13980   9756 S   0.0   1.5   0:07.15 systemd
      2 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kthreadd
      3 root      20   0       0      0      0 S   0.0   0.0   0:00.00 pool_workqueue_release
      4 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-rcu_gp
      5 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-sync_wq
      6 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-kvfree_rcu_reclaim
      7 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-slub_flushwq
      8 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-netns
      9 root      20   0       0      0      0 I   0.0   0.0   0:00.06 kworker/0:0-events
     10 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/0:0H-events_highpri
     11 root      20   0       0      0      0 I   0.0   0.0   0:00.52 kworker/u8:0-events_power_efficient
     12 root      20   0       0      0      0 I   0.0   0.0   0:00.11 kworker/0:1-events
     13 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-mm_percpu_wq
     14 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_rude_kthread
     15 root      20   0       0      0      0 I   0.0   0.0   0:00.00 rcu_tasks_trace_kthread
     16 root      20   0       0      0      0 S   0.0   0.0   0:00.10 ksoftirqd/0
     17 root      20   0       0      0      0 I   0.0   0.0   0:00.31 rcu_sched
     18 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_par_gp_kthread_worker/0
     19 root      20   0       0      0      0 S   0.0   0.0   0:00.00 rcu_exp_gp_kthread_worker
     20 root      rt   0       0      0      0 S   0.0   0.0   0:00.00 migration/0
     21 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/0
     22 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/0
     23 root      20   0       0      0      0 S   0.0   0.0   0:00.00 cpuhp/1
     24 root     -51   0       0      0      0 S   0.0   0.0   0:00.00 idle_inject/1
     25 root      rt   0       0      0      0 S   0.0   0.0   0:00.06 migration/1
     26 root      20   0       0      0      0 S   0.0   0.0   0:00.12 ksoftirqd/1
     28 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/1:0H-events_highpri
     29 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kdevtmpfs
     30 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-inet_frag_wq
     31 root      20   0       0      0      0 S   0.0   0.0   0:00.00 kauditd
     32 root      20   0       0      0      0 S   0.0   0.0   0:00.00 khungtaskd
     34 root      20   0       0      0      0 S   0.0   0.0   0:00.00 oom_reaper
     35 root      20   0       0      0      0 I   0.0   0.0   0:00.28 kworker/u8:2-events_power_efficient
     36 root       0 -20       0      0      0 I   0.0   0.0   0:00.00 kworker/R-writeback
     37 root      20   0       0      0      0 S   0.0   0.0   0:00.11 kcompactd0
     38 root      25   5       0      0      0 S   0.0   0.0   0:00.00 ksmd
     39 root      39  19       0      0      0 S   0.0   0.0   0:00.00 khugepaged
```

15.Stop the instance

Conclusion:

In conclusion, Amazon EC2 is a powerful IaaS solution that provides scalable, secure, and cost-effective virtual computing resources. Its flexibility, high availability, and seamless integration with Amazon Web Services services make it an essential tool for building and managing cloud-based applications.